

Taş Ev (The Stone House)

Arben, Gjirokastrë'e hâkim tepelerde güzel bir arazi miras almıştır. Şehrin ünlü taş çatılı evlerinden ilham alarak kendine geleneksel bir ev inşa etmeyi planlamaktadır.

Arazi, satırları ve sütunları 1'den N 'ye kadar numaralandırılmış $N \times N$ boyutunda bir kare ızgara olarak temsil edilmektedir. Kayalık arazinin tuhaf bir özelliği vardır: ızgaranın her satırında ve her sütununda tam olarak bir tane büyük, yerinden oynatılamaz kaya vardır.

Evinin temelini atmak için Arben, hiç kaya içermeyen $S \times S$ boyutunda bir bölge olan bir **boş kare alt ızgara** seçmelidir.

M , yukarıdaki koşulları sağlayan tüm kaya dizilimleri için $M \times M$ boyutunda boş bir kare alt ızgaranın var olmasının garanti olduğu en büyük tam sayı olsun. Arben, boyutu en az $M \times M$ olan *herhangi bir* boş kare alt ızgara bulmak istemektedir.

Araziyi incelemek için Arben, arkadaşı Besa'dan yardım ister. Besa, **Gurëmatës** ("taş sayacı") adı verilen özel bir ölçüm cihazı getirmiştir.

Tek bir işlemde Besa, Gurëmatës'i sol üst köşesi (x_1, y_1) ve sağ alt köşesi (x_2, y_2) ile verilen dikdörtgen bir alt ızgarayı taramak üzere ayarlayabilir. Cihaz, o bölgedeki kayaların tam sayısını döndürür. Gurëmatës küçük bir pille çalışmaktadır ve her tarama çok fazla enerji tüketmektedir; bu nedenle Besa cihazı mümkün olduğunca az kullanmalıdır.

Gurëmatës'i kullanarak $S \geq M$ olacak şekilde $S \times S$ boyutunda boş bir kare alt ızgara bulan ve konumunu bildiren bir program yazınız.

Uygulama ayrıntıları

Aşağıdaki prosedürü kodlamalısınız:

```
std::vector < int > solve(int N);
```

- N : ızgaranın boyutu.
- Prosedür tam olarak dört tam sayı $\{x_1, y_1, x_2, y_2\}$ döndürmelidir: bulduğu boş kare alt ızgaranın sol üst ve sağ alt köşeleri. $S \geq M$ olmak üzere $x_2 - x_1 = y_2 - y_1 = S - 1$ koşulu sağlanmalıdır.
- Bu prosedür her test durumu için tam olarak bir kez çağrılır.

Prosedür, aşağıdaki prosedüre çağrı yapabilir:

```
int ask(int x1, int y1, int x2, int y2);
```

- x_1, y_1 : taranan alt ızgaranın sol üst köşesinin satırı ve sütunu.
- x_2, y_2 : taranan alt ızgaranın sağ alt köşesinin satırı ve sütunu.
- $1 \leq x_1 \leq x_2 \leq N$ ve $1 \leq y_1 \leq y_2 \leq N$ koşulları sağlanmalıdır. Aksi takdirde programınız derhal sonlandırılır ve o test durumu için 0 puan alır.
- Prosedür, taranan alt ızgaradaki kayaların sayısını döndürür.

Değerlendirici uyarlanabilir (adaptif) değildir: kayaların konumları `solve` çağrılmadan önce sabitlenmiştir ve `ask` çağrılarınıza bağlı değildir.

Kısıtlar

- $2 \leq N \leq 500$
- `ask` prosedürüne en fazla $2.5 \cdot 10^5$ çağrı yapabilirsiniz.

Alt görevler

Alt görev	Puan	Ek kısıtlar
1	15	$N \leq 40$
2	25	$N \leq 150$
3	60	Ek kısıt yoktur.

Puanlama

`solve` geçersiz bir bölge döndürürse, `ask` prosedürüne geçersiz bir çağrı yaparsa veya `ask` prosedürünü $2.5 \cdot 10^5$ kereden fazla çağırırsa, o test durumu için 0 puan alırsınız.

Aksi takdirde Q , `ask` çağrılarının sayısı olsun. O test durumu için, alt görevin puanının $\min(1, f(Q))$ ile çarpımını alırsınız; burada

$$f(Q) = \begin{cases} \max(0, -0.05 \cdot Q + 1.55), & Q \leq 15 \text{ ise} \\ \max(0, -0.19 \cdot \log_{10}(Q) + 1.02), & Q > 15 \text{ ise} \end{cases}$$

Bir alt görevdeki puanınız, o alt görevdeki tüm test durumları arasındaki en düşük puandır. Nihai puan, alt görevlerin puanlarının toplamıdır.

Puanınızı tahmin etmenize yardımcı olmak için aşağıdaki tablo, bazı belirli Q değerleri için yaklaşık çarpanı göstermektedir:

Sorgu sayısı (Q)	Çarpan ($\approx$)
$Q \leq 11$	1.00
12	0.95
15	0.80
50	0.70
100	0.64
1000	0.45
10000	0.26
100000	0.07
$Q \geq 233573$	0.00

Örnek değerlendirici

Örnek değerlendirici girdiyi aşağıdaki biçimde okur:

```
N
c_1 c_2 ... c_N
```

Burada c_i , i . satırdaki kayanın sütunudur ($1 \leq c_i \leq N$).

Örnek değerlendirici, sorgularınızla kazanılan $\min(1, f(Q))$ çarpanını içeren tek bir satır yazdırır; döndürülen alt ızgara veya herhangi bir ask çağırısı geçersizse 0 yazdırır.

Örnekler

Örnek 1

Örnek değerlendirici girdisi:

```
4
1 2 3 4
```

Kayalar (1, 1), (2, 2), (3, 3) ve (4, 4) konumlarındadır. Programınız bunu bilmemektedir.

Değerlendirici `solve(4)` çağrısını yapar. Olası bir çağrı dizisi şöyledir:

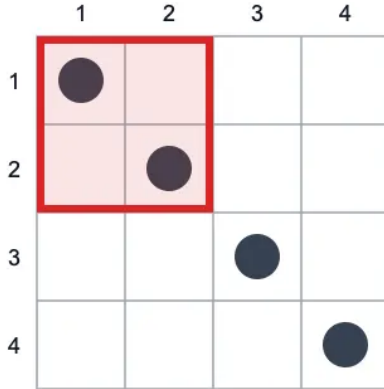
- `ask(1, 1, 2, 2)` çağrısı 2 döndürür.
- `ask(2, 4, 4, 4)` çağrısı 1 döndürür.
- `ask(3, 1, 3, 2)` çağrısı 0 döndürür.

Ardından `solve {1, 3, 2, 4}` döndürür.

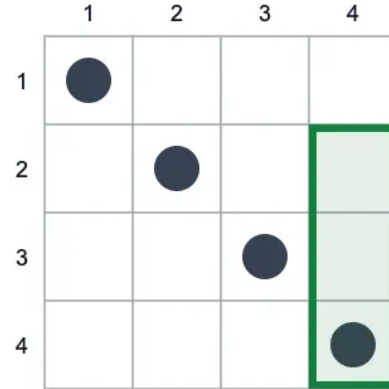
Örnek değerlendirici çıktısı:

```
1
```

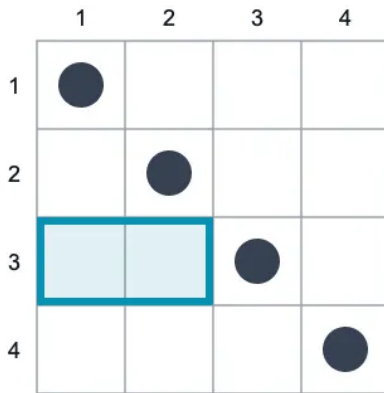
İlk çağrı sol üstteki 2×2 bloğu tarar ve (1, 1) ile (2, 2) konumlarındaki kayaları bulur. İkincisi 4. sütunun 2 ile 4 arasındaki satırlarını tarar ve (4, 4) konumundaki kayayı bulur. Üçüncüsü 3. satırın ilk iki hücrelerini tarar ve hiç kaya bulmaz. Döndürülen bölge 1 ile 2 arasındaki satırları ve 3 ile 4 arasındaki sütunları kapsar ve bu, boş bir 2×2 alt ızgaradır.



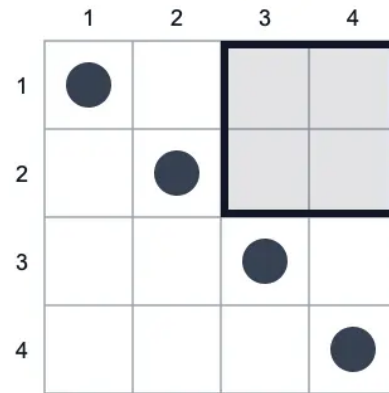
ask(1, 1, 2, 2) returns 2



ask(2, 4, 4, 4) returns 1



ask(3, 1, 3, 2) returns 0



return {1, 3, 2, 4}

$N = 4$ için program $Q = 3$ çağrı kullanmıştır; dolayısıyla $f(Q) = 1.4$ olur ve çarpan 1'dir. Başka cevaplar da kabul edilir; örneğin $\{3, 1, 4, 2\}, \{1, 2, 1, 2\}, \{3, 2, 3, 2\}, \{3, 4, 3, 4\}$ veya $N = 4$ için $M = 1$ olduğundan herhangi bir boş hücre.